

연관규칙

"A를 산 사람은 B도 사더라"를
영수증 더미에서 자동으로 찾아내는 방법.

Q1 무엇을 찾는 건가

편의점 영수증을 1만 장 모아놓고 봤더니, 라면을 산 사람 중 상당수가 계란도 같이 샀습니다.

라면 → 계란

이런 걸 **규칙**이라고 부릅니다. 마트에서 라면 옆에 계란을 두는 이유죠.



사람이 일일이 세는 대신, 컴퓨터가 모든 조합을 다 세어줍니다

Q2 숫자 세 개만 알면 됩니다

영수증 100장이 있다고 해봅시다. 라면은 25장, 계란은 40장에 등장했고, 둘이 같이 나온 건 20장입니다.

지지도 둘 다 같이 산 게 전체의 몇 %?



→ 너무 작으면 어쩌다 한 번 일어난 우연입니다

신뢰도 라면 산 사람 중 계란도 산 비율



→ 높으면 좋아 보이지만... 함정이 있습니다

항상도 이 규칙, 진짜 의미 있나?

신뢰도 0.8 ÷ 계란을 그냥 살 확률 0.4

= 2.0

→ 라면을 사면 계란 살 확률이 **2배**로 올라간다

항상도가 제일 중요합니다.

- 1보다 크면 — 진짜 관계가 있음
- 1이면 — 우연. 아무 상관 없음
- 1보다 작으면 — 오히려 같이 안 삼

Q3 신뢰도만 보면 속습니다

비닐봉투는 원래 손님의 90%가 삽니다. 그럼 이런 규칙이 나와요.

"무엇을 사든 → 비닐봉투" 신뢰도 90%!

그런데 비닐봉투를 그냥 살 확률도 90%입니다

항상도 = $0.9 \div 0.9 = 1.0$

→ 규칙이 아니라 그냥 봉투가 잘 팔리는 것. 자동으로 걸러집니다.

항상도는 "원래 잘 팔리는 것" 효과를 빼줍니다. 그래서 진짜 관계만 남아요.

편의점 영수증 20장

기준을 낮추면 규칙이 쏟아지고, 높이면 알짜만 남습니다. 둘 다 움직여보세요.

최소 지지도 — 이만큼은 같이 팔려야 함 0.15

최소 향상도 — 이만큼은 의미 있어야 함 1.00

찾은 규칙 수

10개

~를 사면	~도 산다	지지도	신뢰도	향상도
우유	빵	0.20	0.57	2.29
콜라	과자	0.20	0.57	2.29
빵	우유	0.20	0.80	2.29
과자	콜라	0.20	0.80	2.29
라면	계란	0.40	0.73	1.45
계란	라면	0.40	0.80	1.45
계란	우유	0.20	0.40	1.14
우유	계란	0.20	0.57	1.14
라면	콜라	0.20	0.36	1.04
콜라	라면	0.20	0.57	1.04

적당한 개수입니다. 향상도가 높은 위쪽부터 보면 됩니다.

규칙이 너무 많이 나오면 사람이 다 볼 수가 없습니다. 기준을 올려서 알짜만 남기는 게 실제로 하는 일 이에요.

Q5 코드로 보기

```
# pip install mlxtend
import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules

receipts = [
    ['라면', '계란', '우유'],
    ['라면', '계란'],
    ['라면', '계란', '콜라'],
    ['우유', '빵'],
    ['라면', '콜라'],
]

# ① 표로 바꾸기 - 싶으면 True
items = sorted({i for r in receipts for i in r})
df = pd.DataFrame([i: (i in r) for i in items} for r in receipts])

# ② 자주 같이 나오는 조합 찾기
freq = apriori(df, min_support=0.4, use_colnames=True)

# ③ 규칙 뽑기 - 향상도 1 이상만
rules = association_rules(freq, metric='lift', min_threshold=1.0)
print(rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']])
```

결과 읽는 법

`antecedents` 는 "~를 사면" 쪽, `consequents` 는 "~도 산다" 쪽입니다.

(라면) → (계란), 신뢰도 0.75, 향상도 1.25 라면

"라면 산 사람의 75%가 계란도 샀고, 그냥 계란 살 확률보다 1.25배 높다"는 뜻이에요.

`min_support` 를 너무 낮추면 규칙이 폭발적으로 늘어납니다. 처음엔 **0.1~0.4** 사이에서 시작해서 조정해보세요.

Q6 어디에 쓰나

분 야	쓰 임
마트	진열 위치 결정, 세트상품 기획, 묶음 할인
쇼핑몰	"이 상품을 본 사람들이 함께 본 상품"
병원	같이 나타나는 증상 패턴 찾기
웹사이트	같이 클릭되는 메뉴 조합 → 화면 배치 개선

인과관계가 아닙니다

"라면 → 계란"은 **라면 때문에 계란을 샀다는 뜻이 아니에요.** 그냥 같이 산다는 관찰일 뿐입니다.
아이스크림과 선풍기가 같이 팔리는 건 **둘 다 더워서지**, 아이스크림이 선풍기를 부른 게 아니듯이요.

Q7 한 장 정리

지 표	질 문	주 의
지지도	얼마나 자주 같이 사나	너무 낮으면 우연
신뢰도	A를 산 사람 중 몇 %가 B도 사나	단독으로 보면 속는다
향상도	이 규칙이 진짜 의미 있나	1을 기준으로 판단

이것만 기억하면 됩니다

- ① 정답이 없는 데이터에서 **숨은 패턴을 발견하는 일** → 비지도학습
- ② 숫자 세 개 중 **향상도가 왕이다**
- ③ 향상도 1은 **"아무 상관 없음"**이라는 뜻
- ④ 같이 팔린다는 것이 **원인이라는 뜻은 아니다**

Q9 Level별 코드 작성하기

옆에 주피터 노트북을 열고, 복사 버튼으로 코드를 붙여넣어 따라 해보세요. 쉬운 것부터 실전(캐글 공공데이터)까지 3단계입니다.

코드 해석 ▼을 누르면 한 줄씩 쉽게 풀어 줍니다.

(연관규칙은 `pip install mlxtend` 가 필요해요.)

LEVEL 1 단순 — 영수증 5장에서 규칙 찾기

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

data = [['라면', '계란'], ['라면', '계란', '콜라'], ['라면', '콜라'], ['우유', '빵'], ['라면', '계란']]
te = TransactionEncoder()
df = pd.DataFrame(te.fit(data).transform(data), columns=te.columns_) # 0/1 표로

freq = apriori(df, min_support=0.4, use_colnames=True) # 자주 나온 조합
rules = association_rules(freq, metric='confidence', min_threshold=0.6)
print(rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']])
```

```
import ... TransactionEncoder / apriori / association_rules
```

표 다루기(pandas)와 연관규칙 도구(mlxtend)를 가져옵니다.

```
data = [['라면', '계란'], ...]
```

영수증 5장이에요. 한 줄이 한 번의 장바구니(산 물건 목록)입니다.

```
df = pd.DataFrame(te.fit(data).transform(data), ...)
```

글자 목록을 **샀으면 True/안 샀으면 False**인 0/1 표로 바꿉니다(컴퓨터가 계산하기 쉽게).

```
freq = apriori(df, min_support=0.4, ...)
```

전체의 **40% 이상** 등장한 '자주 팔린 조합'만 골라냅니다(드문 건 규칙으로 쓸모없어서 미리 버림).

```
rules = association_rules(freq, metric='confidence', min_threshold=0.6)
```

그 조합에서 'A를 사면 B도 산다' 규칙을 뽑되, **신뢰도 0.6 이상**만 남깁니다.

```
print(rules[['...']])
```

규칙과 지지도·신뢰도·**향상도(lift)**를 봅니다. lift가 1보다 크면 진짜 관계!

LEVEL 2 복잡한 데이터 — 거래·품목 늘리고 향상도 순 정렬

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

data = [
    ['우유', '빵', '버터'], ['빵', '버터'], ['우유', '빵'], ['우유', '빵', '버터', '계란'],
    ['빵', '계란'], ['우유', '버터'], ['빵', '버터', '계란'], ['우유', '빵', '계란'],
]
te = TransactionEncoder()
df = pd.DataFrame(te.fit(data).transform(data), columns=te.columns_)

freq = apriori(df, min_support=0.3, use_colnames=True)
rules = association_rules(freq, metric='lift', min_threshold=1.0)
rules = rules.sort_values('lift', ascending=False) # 향상도 높은 순
print(rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']].head())
```

```
data = [ ... 8장 ... ]
```

영수증을 8장으로 늘리고 품목도 늘렸어요. 데이터가 많을수록 규칙이 더 그럴듯해집니다.

```
apriori(df, min_support=0.3, ...)
```

데이터가 늘어 기준(min_support)을 0.3으로 살짝 낮춰 더 많은 조합을 봅니다.

```
association_rules(freq, metric='lift', min_threshold=1.0)
```

이번엔 **향상도(lift) 1 이상**인 규칙만 — 즉 '진짜 관계 있는' 규칙만 뽑습니다.

```
rules.sort_values('lift', ascending=False)
```

향상도가 **높은 순으로 정렬**합니다. 위쪽이 가장 강한 관계예요.

```
...head()
```

상위 몇 개만 봅니다(규칙이 많을 때 알짜만 보기).

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

# 공개 미리 URL로 바로 불러오기 (Kaggle 'Groceries' - 마트 거래 9835건)
import urllib.request
url = 'https://raw.githubusercontent.com/stedy/Machine-Learning-with-R-datasets/master/groceries.csv'
lines = urllib.request.urlopen(url).read().decode('utf-8').strip().splitlines()
data = [ln.split(',') for ln in lines]          # 한 줄 = 한 번의 장바구니

te = TransactionEncoder()
df = pd.DataFrame(te.fit(data).transform(data), columns=te.columns_)

freq = apriori(df, min_support=0.02, use_colnames=True)          # 2% 이상 등장
rules = association_rules(freq, metric='confidence', min_threshold=0.3)
rules = rules.sort_values('lift', ascending=False)
print(rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']].head())
```

```
url = '...groceries.csv' → urllib.request.urlopen(url)...
```

마트 거래 **9835건**을 **공개 URL에서 바로** 줄 단위로 읽습니다(다운로드 불필요). 거래마다 품목 수가 달라 표 대신 이렇게 읽어요.

```
data = [ln.split(',') for ln in lines]
```

각 줄을 쉼표로 쪼개 '장바구니 목록'으로 만듭니다.

```
apriori(df, min_support=0.02, ...)
```

거래·품목이 많아 기준을 **2%**로 확 낮춥니다(안 그러면 규칙이 거의 안 남아요).

```
association_rules(..., min_threshold=0.3)
```

신뢰도 0.3 이상 규칙을 뽑고, 향상도 높은 순으로 정렬해 상위를 봅니다.

결과 예

'뿌리채소+우유 → 기타채소'처럼 **향상도 2배 이상**인 진짜 강한 조합이 나와요. 이런 걸 매대 배치·추천에 씁니다.